Why DeployGuard Standardized Builds on Railpack

Architecture Defense: Why We Replaced Static Dockerfile Logic and Ignore Repository Dockerfiles as Build Authority

Architectural position
DeployGuard intentionally uses Railpack as the single source-to-image build authority for supported applications. We no longer maintain platform-owned static Dockerfile templates, and repository-owned Dockerfiles are not treated as an alternative build path. DeployGuard owns admission and the application contract; Railpack owns how supported source code is converted into a runnable image.

1. Why the original static-Dockerfile approach was not sustainable

At the beginning, generating our own Dockerfiles appeared attractive because it gave us direct control over the build. That approach works for a small, known set of application shapes. It becomes fragile when the platform must accept repositories created by different developers, languages, package managers, frameworks, and directory structures.

- A static template encodes assumptions: one runtime image, one dependency installer, one build command, one start command, one directory layout, and one set of system packages.
- Node.js alone can involve npm, pnpm, yarn, bun, workspaces, monorepos, native modules, framework build outputs, and different start conventions.
- Python introduces requirements.txt/pyproject variants, binary wheels, source builds, compiler requirements, native libraries, and packages such as psycopg2 that can depend on system capabilities.
- Every new edge case would force DeployGuard to add another detector, Dockerfile branch, compatibility exception, or framework-specific fix. Over time, DeployGuard would effectively become a second build system.
- This also made failures harder to classify: was the repository invalid, was our detector wrong, was the generated Dockerfile wrong, or was the runtime itself unsupported?

The problem was therefore not Docker itself. The problem was making DeployGuard responsible for generating correct Docker build instructions for arbitrary repositories.

2. Why we also do not use repository-owned Dockerfiles as the default authority

Allowing every repository Dockerfile to become authoritative would solve the template-maintenance problem by transferring build behavior to each repository, but it would create a different architectural problem: DeployGuard would lose a uniform, predictable build contract.

- Two repositories using the same language could build in completely different ways, making platform behavior much harder to validate and certify.
- A Dockerfile can introduce arbitrary base images, hidden system dependencies, custom package repositories, privileged assumptions, undeclared network behavior, and framework-specific logic outside DeployGuard's compatibility model.
- DeployGuard would need two parallel build authorities - Railpack for some projects and arbitrary Dockerfiles for others - which increases state, testing, diagnostics, and support complexity.
- A general PaaS benefits from one normal build path. Unsupported or special repositories should fail with a clear compatibility decision rather than silently switching to a second uncontrolled build model.

Defense point: We did not replace our Dockerfiles with user Dockerfiles. We removed competing Dockerfile authorities and standardized supported builds through one specialized builder: Railpack.

3. The exact responsibility boundary

Responsibility	Owner	Why
Select exact repository, branch, commit SHA, service and directory	DeployGuard	Defines exactly what source is admitted.
BuildTarget and repository-structure admission	DeployGuard	Rejects ambiguity and unsupported layouts before trusting a build.
Compatibility policy	DeployGuard	Decides whether the repository is within the platform's supported envelope.
Port, HOST and required runtime environment contract	DeployGuard	Defines observable behavior the produced application must satisfy.
Reserved environment and secret ownership	DeployGuard	Protects platform-owned runtime values and configuration boundaries.
Language/runtime analysis for building	Railpack	Specialized application-source build intelligence.
Dependency installation strategy	Railpack	Determines how supported application dependencies are installed.
Build steps and start-command generation	Railpack	Produces the build plan from source and configuration.
OCI/container image construction	Railpack	Turns the supported build plan into a runnable image.
Validate the result against the admitted application contract	DeployGuard	A successful image build is not enough; it must still satisfy the platform contract.
Deployment lifecycle and release authority	DeployGuard	Build technology does not decide lifecycle or application validity.

4. What happens with dependency problems such as psycopg2?

The psycopg2 example demonstrates why the boundary matters. A Python application may be structurally valid while one dependency requires native PostgreSQL build capability. The correct solution is not to add a permanent DeployGuard Dockerfile special case for every package name.

- Railpack owns the mechanics of installing dependencies and constructing the supported build environment.
- DeployGuard owns the compatibility decision: if the platform promises that capability, failure to provide it can become a platform defect; if the repository requires an unsupported or undeclared native capability, DeployGuard should classify it as a compatibility blocker.
- This keeps the system capability-oriented instead of growing an endless list such as psycopg2 -> special Dockerfile, sharp -> special Dockerfile, canvas -> special Dockerfile, and so on.

5. Why this is a stronger PaaS architecture

- Single build authority: supported applications follow one normal source-to-image path instead of static templates plus arbitrary repository Dockerfiles.
- Clear separation of concerns: DeployGuard decides what is admissible and what the resulting application must do; Railpack specializes in how supported source is built.
- Lower maintenance risk: framework/package-manager evolution belongs primarily to the specialized build layer rather than forcing repeated DeployGuard Dockerfile rewrites.
- Better certification: one build model is easier to test, reason about and defend than multiple competing build authorities.

Final defense statement

DeployGuard did not give away its core responsibility to Railpack. We deliberately stopped implementing a second application build system. Railpack is the specialized build engine; DeployGuard is the authority that selects the exact source, decides whether it is supported, defines the application contract, validates the build result, and controls the application lifecycle. Standardizing supported builds through Railpack gives us one predictable and certifiable build path instead of maintaining static Dockerfiles or accepting arbitrary repository Dockerfiles as competing authorities.

Reference basis: DeployGuard permanent invariant: Railpack remains the authority for how repository application code is built and started; current Railpack path does not use generated Dockerfile/build-plan payloads as legacy authorities.